

SOS Dev

Quand l'IA ne suffit plus, un développeur te rejoint depuis ton terminal.

Une plateforme d'entraide qui vient te chercher là où tu bloques : dans ton terminal. Une commande, et SOS Dev cherche d'abord une solution déjà connue, puis une piste de l'IA. Si ça ne suffit pas, ton erreur et ton code (sans tes secrets) partent vers un développeur disponible qui connaît ta technologie. Il te rejoint dans une session partagée reliée à ta machine : il propose, tu valides, tu relances. À la fin, la session devient une solution que tout le monde peut retrouver.

Nouveautés de la v2 (surlignées en vert dans les tableaux) : boucle « Fiches → IA → Humain », terminal partagé avec retour des corrections sur la machine, règles de mise en relation précises, confidentialité « effacé à la fin de la session », garde-fous anti-abus, plan ramené à 3 semaines, stack simplifiée.

Le problème que SOS Dev résout

Le problème	Ce que ça cause
Demander de l'aide coûte du temps	Il faut quitter son outil, expliquer le contexte, copier le code, rédiger une question. Beaucoup abandonnent.
La réponse arrive tard	Sur un forum ou un groupe, on attend des heures ou des jours.
L'IA ne suffit pas toujours	Les assistants IA répondent vite, mais tournent en rond sur les vrais blocages : contexte du projet, environnement, bug subtil. Il manque un humain.
L'aide est écrite, pas vécue	Un texte ne montre pas comment l'expert a cherché.
Le savoir se perd	Une explication en message privé disparaît. La même question revient.
Les aidants ne sont pas trouvés	Les développeurs prêts à aider ne savent pas qui a besoin d'eux, ni quand.

Pourquoi pas juste ChatGPT ou Copilot ?

SOS Dev ne remplace pas l'IA : il prend le relais quand elle échoue. La commande **sos** cherche d'abord dans la bibliothèque de fiches, puis propose une piste de l'IA (environ 10 secondes). Seulement si ça ne suffit pas, un humain est appelé, et il reçoit ce que l'IA a déjà essayé. Résultat : les humains ne traitent que les vrais blocages, et chacun d'eux devient un savoir vérifié par deux développeurs, ce qu'aucune IA ne produit seule.

Notre solution : la boucle SOS

- 1 Capturer**
sos npm start lance le programme et récupère l'erreur, les fichiers cités dans la trace, les fichiers de dépendances, la version du langage et l'OS.
- 2 Protéger**
Secrets masqués sur ta machine avant tout envoi. Tu vois l'aperçu, tu coches ou décoches les fichiers, tu valides.
- 3 Chercher d'abord**
Fiches déjà résolues similaires, puis piste de l'IA. Si ça règle le problème : terminé en quelques secondes.

4 Alerter un humain

Sinon, bouton « Appeler un humain » : les aidants disponibles qui connaissent ta technologie sont alertés. Le premier qui accepte te rejoint.

5 Résoudre ensemble

Salle SOS : code partagé en direct, chat, et ton terminal diffusé en lecture seule. Les corrections de l'aidant arrivent chez toi sous forme de diff que tu valides, puis tu relances.

6 Capitaliser

L'IA rédige un brouillon de fiche (erreur, cause, correction). Les deux personnes relisent et valident. La fiche rejoint la bibliothèque et le profil de l'aidant.

Le terminal partagé (cœur de la démo)

Pendant la session, la commande **sos** reste ouverte et fait le pont entre la salle et la machine du demandeur :

Fonction	Comment
Sortie en direct	Chaque relance du programme s'affiche dans la salle, en lecture seule pour l'aidant.
Retour des corrections	Quand l'aidant modifie un fichier, le demandeur voit le diff dans son terminal et répond o/n. Seuls les fichiers partagés peuvent être écrits.
Demander une relance	L'aidant clique, mais c'est le demandeur qui appuie sur Entrée. Aucun code ne s'exécute sans son accord.

Règles précises

Sujet	Règle
Fichiers envoyés	Fichiers cités dans la trace + fichiers de dépendances (package.json, requirements.txt...). Limite : 10 fichiers ou 200 Ko. Ajout possible avec <code>sos --add fichier</code> . Toujours exclus : <code>.env*</code> , <code>*.pem</code> , <code>*.key</code> , tout ce qui est dans <code>.gitignore</code> .
Masquage des secrets	Règles écrites par l'équipe (clés connues, mots de passe, jetons) + détection des chaînes aléatoires (entropie). Second masquage sur le chat avant la fiche.
Détection de la techno	Automatique à partir des fichiers du projet, corrigeable par le demandeur.
Mise en relation	Aidants disponibles qui maîtrisent la techno, triés par temps de réponse moyen ; le premier qui accepte rejoint. Après 2 min sans réponse : élargissement aux technos proches (ex. React vers JavaScript). Après 5 min : file publique « en attente », notification dès qu'un aidant la prend.
Garde-fous	Connexion GitHub obligatoire, 3 demandes par heure maximum, note et bouton « signaler » après chaque session, retrait du Radar après 3 signalements.
Motivation des aidants	Profil = portfolio : fiches publiées, badges par techno, à montrer à un recruteur. Les aidants réguliers peuvent devenir mentors rémunérés.

Les écrans principaux

Écran	Contenu
1. Terminal	Commande <code>sos</code> : aperçu de l'envoi, fiches et piste de l'IA, appel d'un humain, puis diffs à valider pendant la session.

Écran	Contenu
2. Radar des aidants	L'aidant se déclare disponible et voit les demandes qui correspondent à ses compétences, avec ce que l'IA a déjà essayé.
3. Salle SOS	Code partagé en direct, chat, terminal du demandeur en lecture seule, bouton « Demander une relance », bouton « Problème résolu ».
4. Fiche solution	Erreur, cause, correction. Recherchable par tout le monde après double validation.
5. Profil	Aides données, temps moyen de résolution, technologies, badges, remerciements reçus.

Fonctionnalités et priorités

Fonction	Ce que ça fait
Commande terminal	Prioritaire. Capture, masquage, aperçu, puis pont pendant la session (sortie en direct, diffs). Repli : coller l'erreur sur le site.
Fiches puis IA d'abord	Prioritaire. Recherche par le sens dans la bibliothèque et piste de l'IA avant d'appeler un humain.
Alerte et mise en relation	Prioritaire. Règle simple et visible, élargissement automatique, file d'attente.
Salle SOS	Prioritaire. Code partagé, chat, terminal partagé. La voix vient après.
Fiche solution automatique	Prioritaire. Brouillon par l'IA, publication après accord des deux, anonymisation possible.
Profil et remerciements	Réputation fondée sur l'aide donnée, badges. Pas de classement toxique.
Voix, Replay, mentorat payant	Phase 2.

Technologies

Toutes sous licence permissive : aucune GPL, AGPL ou LGPL, y compris dans les dépendances indirectes (règlement, article 6). Audit automatique des licences en intégration continue dès la semaine 1.

Besoin	Choix
Commande terminal	Node.js + TypeScript, installable avec npm. Client Yjs pour le pont avec la salle.
Site web	Next.js, React, TypeScript, Tailwind. Installable sur téléphone.
Serveur	NestJS. Alertes en temps réel par WebSocket (Socket.IO).
Code partagé	Yjs + CodeMirror, serveur de synchronisation y-websocket ou Hocuspocus (MIT).
Base de données	PostgreSQL + pgvector (recherche par le sens, doublons).
IA	Appel d'une API d'IA depuis le serveur Node (résumés, embeddings, piste). Plus de service Python séparé. Conditions d'utilisation du modèle vérifiées. À déclarer à la soumission.
Connexion	Auth.js avec GitHub : un clic.
Voix (phase 2)	LiveKit auto-hébergé, ou appel WebRTC simple.

Sécurité et confidentialité

1. Les secrets sont masqués sur la machine de l'utilisateur, avant tout envoi, avec aperçu et validation.
2. Ton code est effacé à la fin de la session (24 h maximum). Seule la fiche validée est gardée. Échanges chiffrés (HTTPS, WSS).
3. Le code n'est vu que par l'aidant de la session, jamais publié.
4. La fiche solution n'est publique qu'avec l'accord des deux personnes, et peut être anonymisée. En cas de refus, elle reste privée dans l'historique des deux.
5. Aucun code n'est exécuté sur nos serveurs. Sur la machine du demandeur, rien n'est écrit ni exécuté sans sa validation explicite (diff o/n, relance par Entrée).

Démonstration devant le jury (2 minutes)

- 1 Une erreur rouge dans le terminal**
L'équipier A lance son programme : il plante.
- 2 Une seule commande**
sos affiche l'aperçu, les secrets sont masqués. Il valide. Aucune fiche ne correspond et la piste de l'IA ne suffit pas : il appelle un humain.
- 3 Un aidant reçoit l'alerte en direct**
L'équipier B voit la demande sur le Radar, avec ce que l'IA a déjà essayé, et accepte.
- 4 Ils corrigent ensemble**
B voit le terminal de A, corrige dans la salle. A valide le diff dans son terminal, relance : tout passe au vert sur les deux écrans.
- 5 Une fiche solution apparaît**
L'IA rédige la fiche, A et B valident. Un nouveau sos sur la même erreur la retrouve instantanément.

Prévoir une vidéo de secours de la démo en cas de problème réseau le jour J.

Pourquoi ça peut gagner (grille de notation)

Critère	Notre atout
Pertinence (25 %)	Résout un vrai problème quotidien, et répond clairement à « pourquoi pas l'IA ? » : SOS Dev prend le relais quand l'IA échoue.
Technique (30 %)	Outil en ligne de commande relié en temps réel à une salle partagée, recherche par le sens, protection des secrets, IA expliquée.
Expérience utilisateur (20 %)	Zéro friction : on reste dans son terminal, une commande, la correction arrive sur sa machine.
Innovation (15 %)	L'entraide va vers le développeur. Chaque aide devient un savoir vérifié par deux humains.
Public (10 %)	Démo spectaculaire : un programme rouge qui repasse au vert sur deux écrans.

Qui paie ?

Les développeurs utilisent gratuitement ; ceux qui en profitent paient.

Qui	Pourquoi
Entreprises	Espace privé pour leurs équipes : aide interne rapide, solutions gardées pour elles.
Écoles et universités	Abonnement pour que leurs étudiants reçoivent de l'aide, avec suivi des progrès.

Qui	Pourquoi
Sponsors	Financement d'une technologie ou d'un événement d'entraide, à leur nom.
Mentorat (option)	Sessions longues avec un aidant reconnu, commission pour la plateforme.

Plan de réalisation (3 semaines)

Quand	Quoi
Semaine 1	Connexion GitHub, base de données, mise en ligne sur Datacloud (WebSockets vérifiés). Commande terminal v1 : capture, masquage, aperçu. Audit des licences en intégration continue.
Semaine 2	Bibliothèque et piste de l'IA, Radar, alertes avec élargissement, Salle SOS : code partagé et chat.
Semaine 3	Terminal partagé (sortie en direct, diffs, relance), fiche solution, profil, données de démonstration, tests, vidéo de secours. Soumission le 30 octobre.

Si on prend du retard, on repousse la voix, le Replay et le mentorat, jamais la boucle complète : commande, fiches et IA, alerte, salle reliée au terminal, fiche solution.

À ne pas oublier (règlement)

1. Si l'équipe gagne, le projet est cédé à Systalink : tout le monde doit être d'accord.
2. Acheter un produit Datacloud pour déployer, avec le code promo de 40 %.
3. Déclarer l'usage de l'IA à la soumission (piste, résumés, recherche).
4. Soumettre avant le 31 octobre à 17 h (heure de Dakar), donc avant 18 h à Cotonou. Objectif interne : le 30.
5. Voter les 1er et 2 novembre : sinon l'équipe est exclue.
6. Pour la démo, prévoir de vrais aidants : les membres de l'équipe, sans faux profils.